



Sistem Programlama

DOÇ. DR. ABDULLAH SEVİN

İçerik

a) Assembler

- <http://web.eecs.utk.edu/~jplank/plank/classes/cs360/360/notes/Assembler3/lecture.html>

Pointers

- ❑ Basit İşaretçi referans kaldırma.
- ❑ Çerçeve işaretçisinden erişilecek olan üç yerel değişken vardır:
- ❑ **i** (fp-8) konumunda . Bu nedenle, &i eşittir (fp-8) ve i'nin değeri [fp-8]'e yüklenecek ve saklanacaktır.
- ❑ Benzer şekilde, &j eşittir (fp-4) ve j'nin değeri [fp-4]'e yüklenecek ve saklanacaktır.
- ❑ Son olarak, &jp, fp'ye eşittir ve jp'nin değeri yüklenecek ve [fp]'de saklanacaktır.
- ❑ *jp istiyorsanız, [fp]'yi bir kayıtçıya yüklemeniz ve kayıtçının referansını kaldırmanız gerekir.

[pointer1.c:](#)

```
int main()
{
    int i, j, *jp;

    jp = &j;
    j = 15;
    i = *jp;
}
```

[pointer1.jas:](#)

```
main:
    push #12                / Allocate the three locals

    mov #-4 -> %r0          / jp = &j.
    add %fp, %r0 -> %r0
    st %r0 -> [fp]

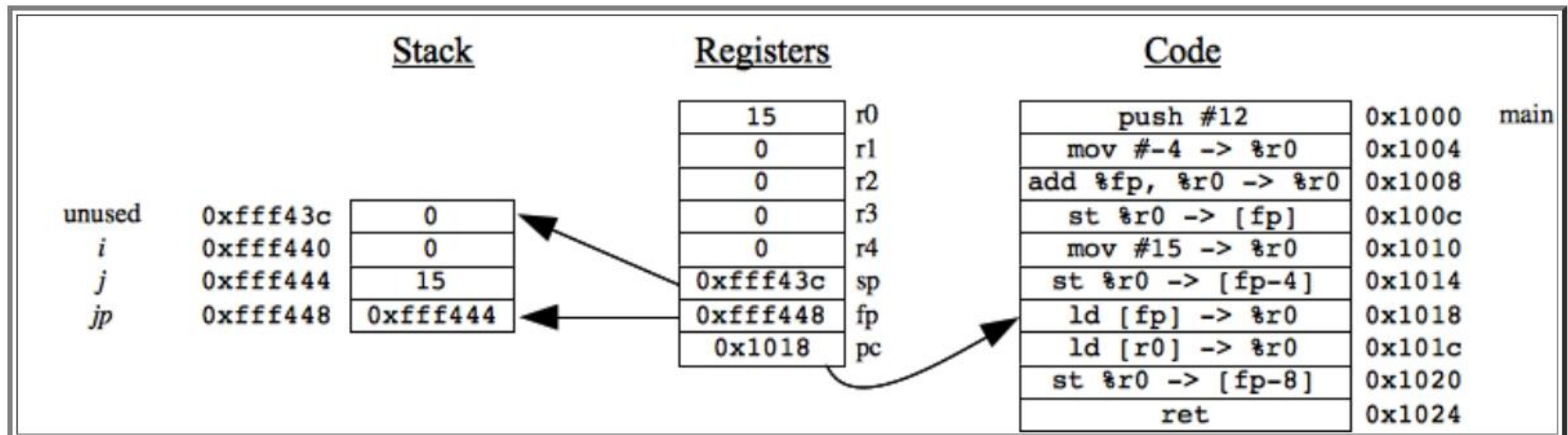
    mov #15 -> %r0          / j = 15
    st %r0 -> [fp-4]

    ld [fp] -> %r0          / i = *jp
    ld [%r0] -> %r0
    st %r0 -> [fp-8]

    ret
```

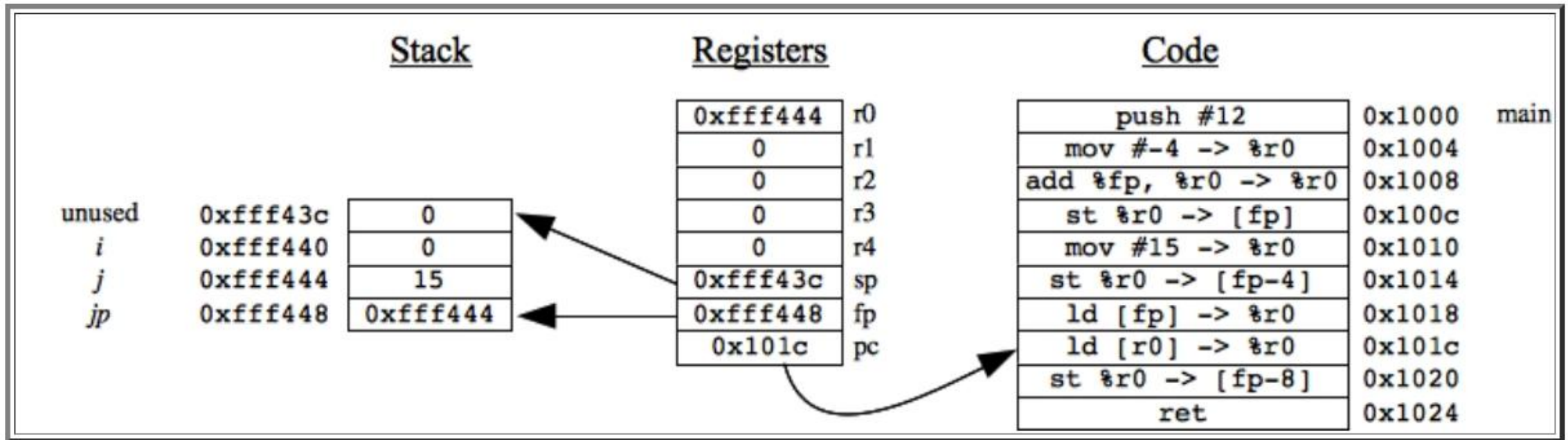

Pointers

İşte programın önemli bir parçası olduğunu düşündüğüm şeyin bir kopyası -- "i = *jp" yapmaya başladığınızda:



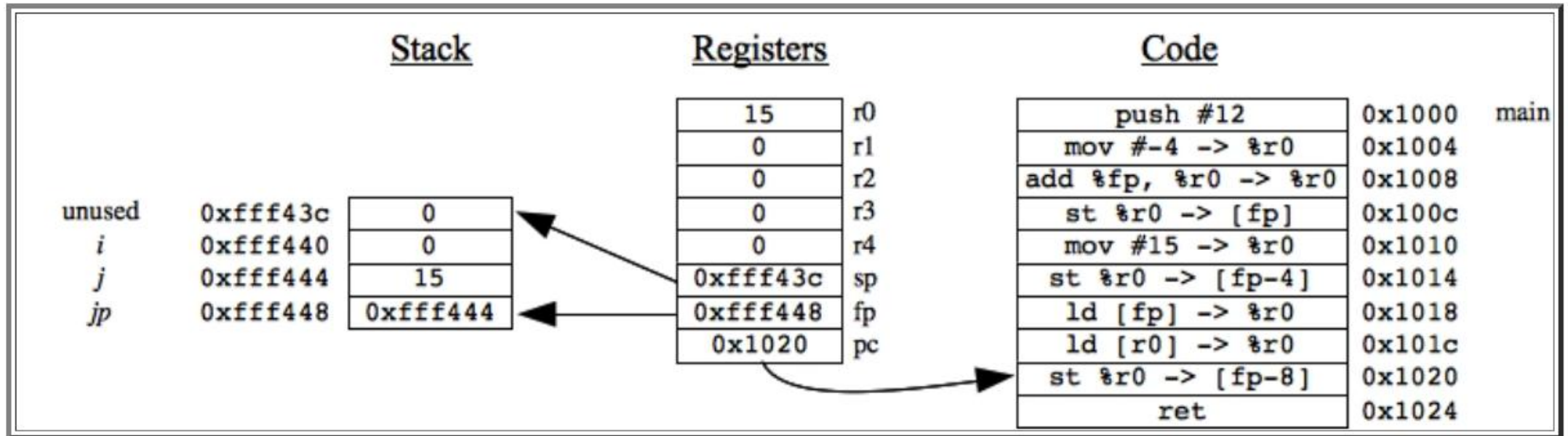
Pointers

- &j'yi manuel olarak 0xffff444 olarak hesaplamak zorunda kaldık. Bu değeri jp = [fp] içinde görebilirsiniz. *jp'yi hesaplamak için önce jp'yi r0'a yüklüyoruz:



Pointers

- Ve sonra [r0] yüklüyoruz: bu, 0xfff444'teki değeri alıyor, yani 15 (jassem'de bunu 0xf olarak görürsünüz, çünkü jassem her şeyi onaltılık-hexadecimal olarak yapar):



- Son olarak, 15, *i*'de depolanır (konum 0xffff440).

Pointers

- ❑ `main()`'de `&i` (`fp-4`)'tür ve `i`'nin değeri `[fp-4]`'tür.
- ❑ `main()`'de `&j`, `fp`'dir ve `j`'nin değeri `[fp]`'dir.
- ❑ `a()`'da `&p` (`fp+12`)'dir ve `p`'nin değeri `[fp+12]`'dir.
- ❑ `*p` elde etmek için `[fp+12]`'yi bir kayda yüklemeniz ve kayıtçının referansını kaldırmanız gerekir.

[pointer2.c:](#)

```
int a(int *p)
{
    return *p;
}

int main()
{
    int i, j;

    j = 15;
    i = a(&j);
}
```

[pointer2.jas:](#)

```
a:
    ld [fp+12] -> %r0      / get p's value
    ld [%r0] -> %r0       / dereference it
    ret

main:
    push #8

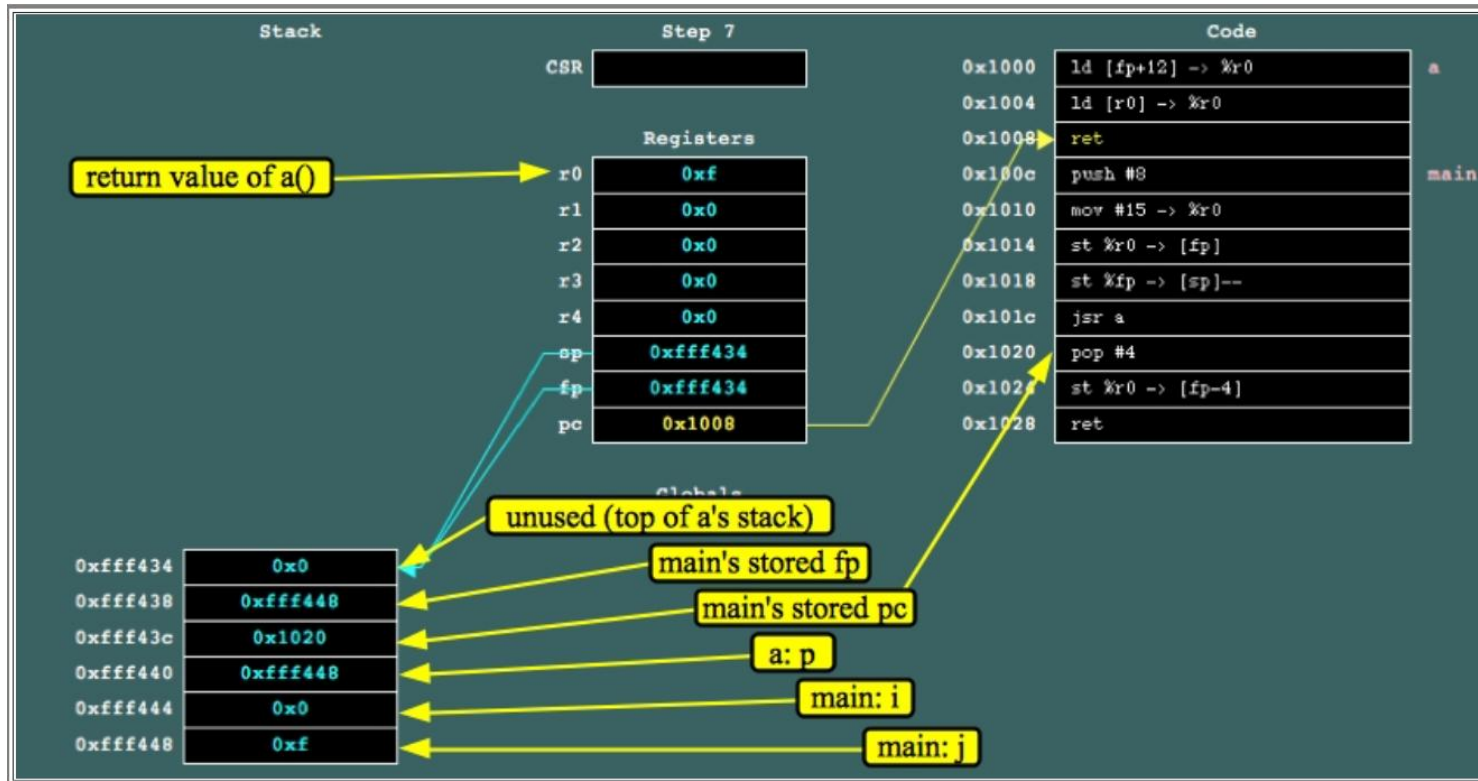
    mov #15 -> %r0        / j = 15
    st %r0 -> [fp]

    st %fp -> [sp]--      / push &j on the stack
    jsr a                 / and call a()
    pop #4
    st %r0 -> [fp-4]

    ret
```

Pointers

- Yığındaki her değeri tanımlayabildiğinizden emin olmamız gerekli. Bunu, kod a() için ret deyimindeyken yapıyoruz:



Dizi Referans kaldırma

[pointer3.c](#):

- ❑ Dizi başvurusunu kaldırma, işaretçi başvurusunu kaldırmaya çok benzer.
- ❑ Dizi indeksini öğenin boyutuyla çarparsınız, ardından onu dizinin en üstüne eklersiniz.
- ❑ Ardından bu değeri kaldırın. Örneğin, pointer3.c'ye bakın:

```
void a(int *p)
{
    int i;

    i = p[0];
    i = p[3];
    i = p[i];
}

int main()
{
    int array[5];

    array[0] = 10;
    array[1] = 11;
    array[2] = 12;
    array[3] = 2;
    array[4] = 15;

    a(array);
}
```

Dizi

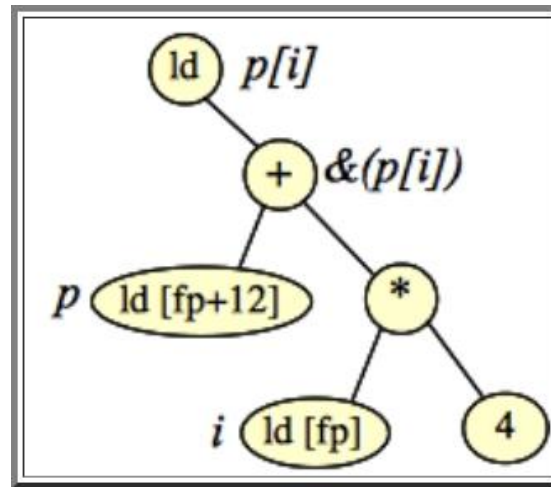
- ❑ Şimdilik main() hakkında endişelenmeyelim. Sadece belleği ayarlamak için main()'i kullanıyoruz böylelikle **a fonksiyonunun** assembler'ını takip edebilirsiniz.
- ❑ Yine, Assmblr'da bulmayı kolaylaştırmak için değişkenlerimizin adreslerini ve değerlerini bulalım:
- ❑ &i fp'dir ve i'nin değeri [fp]'dir.
- ❑ &p (fp+12)'dir ve p'nin değeri [fp+12]'dir.
- ❑ p[0]'ı elde etmek için, [fp+12]'yi yüklememiz ve referansını kaldırmamız gerekiyor.
- ❑ p[3]'ü elde etmek için, [fp+12]'yi yüklememiz, ardından buna 12 eklememiz (3 * bir tamsayının boyutu) ve ardından sonucu kaldırmamız gerekir.
- ❑ p[i]'yi elde etmek için aşağıdaki değeri hesaplamamız gerekir: $[fp+12] + 4 * [fp]$

Dizi

□ Ve sonra referansı kaldırabiliriz. Devam edelim ve referanssızlaştırmayı denkleme koyalım:

• $[fp+12] + 4 * [fp]$

□ Son derste denklemlerle yaptığımız gibi, bunu bir ağaca çevirelim:



Dizi

- $\&(p[i])$ 'yi nasıl oluşturduğumuzu ve sonra onu nasıl kaldırdığımızı görebilmeniz için düğümleri etiketledik . İşte derleyici (pointer3.jas'ta):

```
a:
    push #4

    ld [fp+12] -> %r0      / i = p[0]
    ld [r0] -> %r0
    st %r0 -> [fp]

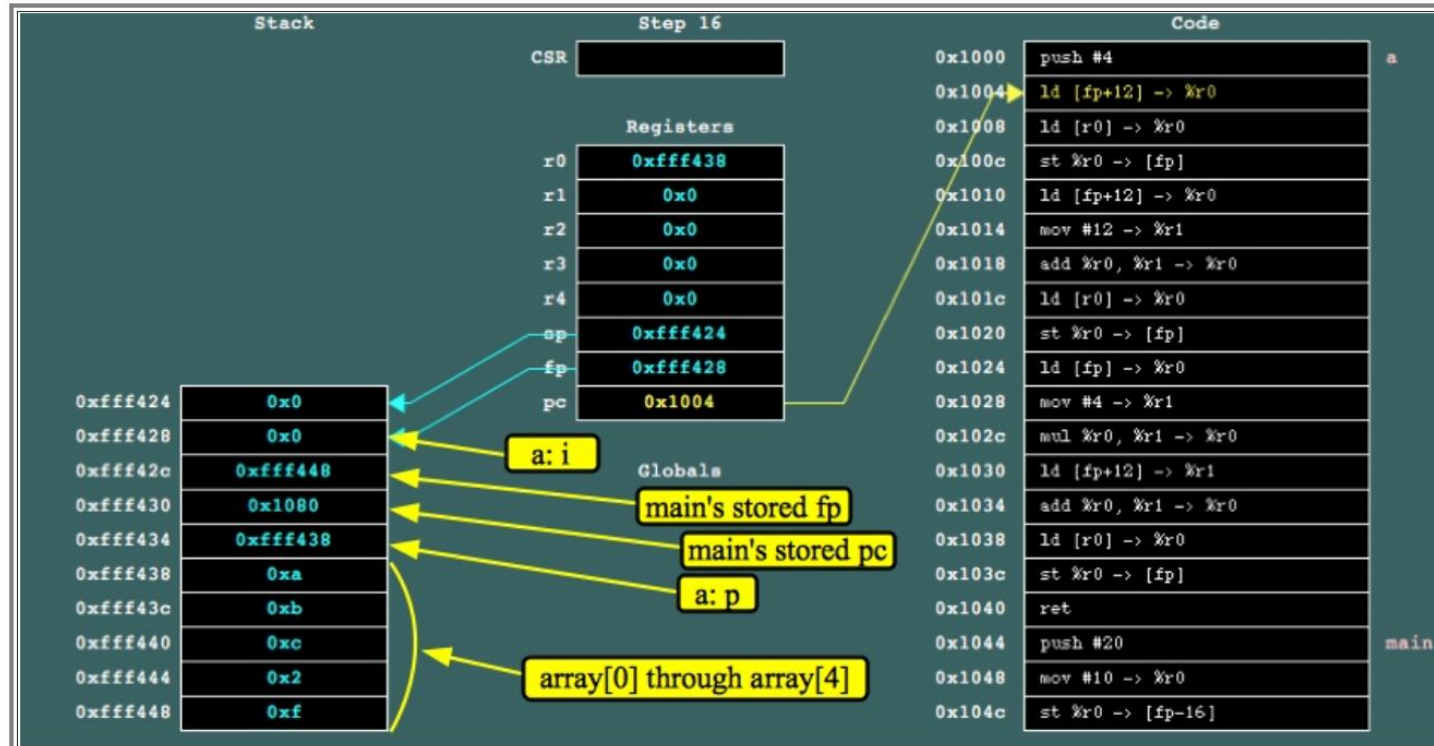
    ld [fp+12] -> %r0      / i = p[3]
    mov #12 -> %r1
    add %r0, %r1 -> %r0
    ld [r0] -> %r0
    st %r0 -> [fp]

    ld [fp] -> %r0         / i = p[i]
    mov #4 -> %r1
    mul %r0, %r1 -> %r0
    ld [fp+12] -> %r1
    add %r0, %r1 -> %r0
    ld [r0] -> %r0
    st %r0 -> [fp]

    ret
```

Dizi

- Yine, şimdilik main()'i görmezden geliyoruz. "Push #4"ten hemen sonra a() çalıştırana kadar jassem'de ilerleyelim. Yine, yığındaki her şeyi tanımlamak yararlıdır:



Dizi

- ❑ `p[0]`'ın 10'a (0xa), `p[3]`'ün 2'ye eşit olduğunu ve `i`'nin 2'ye ayarlandığı için `p[i]`'nin 12'ye (0xc) eşit olduğunu görmelisiniz.
- ❑ Şimdi `main()` hakkında düşünelim. Yapacağı ilk şey, dizinin beş tamsayısını tahsis etmek için `push #20`'yi çağırmak olacaktır. Bundan sonra, derleyici şunu bilir:
- ❑ `&(dizi[0])` (fp-16) ve `dizi[0]`'nin değeri [fp-16]'dır.
- ❑ `&(dizi[1])` (fp-12)'dir ve `dizi[1]`'in değeri [fp-12]'dir.
- ❑ `&(dizi[2])` (fp-8) ve `dizi[2]`'nin değeri [fp-8]'dir.
- ❑ `&(dizi[3])` (fp-4) ve `dizi[3]`'ün değeri [fp-4]'tür.
- ❑ `&(dizi[4])` fp'dir ve `dizi[4]`'ün değeri [fp]'dir.

Dizi

- ❑ Dikkat edeceksiniz -- "array" için ayrılan bir bellek yoktur.
- ❑ Derleyici, "array"ın (fp-16)'ya eşit olduğunu bilir -- bu, dizi[0] için bir işaretçidir.
- ❑ Bu bilgiyle donanmış olarak, dizinin öğelerini ayarlamak basittir.
- ❑ **a(array)** çağırmak biraz daha zordur, ama işte pointer3.jas'ın geri kalanı:
- ❑ a(array)'yi çağırmak için (fp-16)'yı hesaplamalı ve bunu yığına göndermeliyiz.
- ❑ Bu, "mov #-16 -> %r0" ile başlayan üç satırda yapılır.

```
main:
    push #20

    mov #10 -> %r0           / Store the values of array
    st %r0 -> [fp-16]
    mov #11 -> %r0
    st %r0 -> [fp-12]
    mov #12 -> %r0
    st %r0 -> [fp-8]
    mov #2 -> %r0
    st %r0 -> [fp-4]
    mov #15 -> %r0
    st %r0 -> [fp]

    mov #-16 -> %r0          / Push array onto the stack
    add %fp, %r0 -> %r0
    st %r0 -> [sp]--
    jsr a                    / call a
    pop #4
    ret
```

Soru-1

1. Assembly kodu;

```
int a(int i)
{
    int s;
    s=i*i;
    return s;
}

int main()
{
    int i,j;
    i=5;
    j = a(i);

    return 0;
}
```